

ROHINI

COLLEGE OF ENGINEERING & TECHNOLOGY

Approved by AICTE and Affiliated to Anna University (An ISO Certified Institution) | Accredited with A+ Grade by NAAC
Recognized under Section 2(f) of University Grants Commission, UGC ACT 1956

(AUTONOMOUS)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

24CS302-Object Oriented Programming using Java

LAB MANUAL

SEMESTER:III

REGULATION:2021

SUB CODE: 24CS302

SUB NAME: Object Oriented Programming using Java

Prepared By

Approved By

Mrs. Sahila Devi ,AP/CSE

Approved on :

Revision Number:

Revision Date :



ROHINI

COLLEGE OF ENGINEERING & TECHNOLOGY

Approved by AICTE and Affiliated to Anna University (An ISO Certified Institution) | Accredited with A+ Grade by NAAC
Recognized under Section 2(f) of University Grants Commission, UGC ACT 1956
(AUTONOMOUS)

Vision

1. To be an academic institute of continuous excellence towards education and research in rural regime and provide service to nation in terms of nurturing potentially higher social, ethical and engineering companion graduands.

MISSION

2. To foster and promote technically competent graduands by imparting the state of art Engineering education in rural regime.
3. To enunciate research assisted scientific learning by dissemination of knowledge towards science, agriculture, industry and national security.

DEPARTMENT OF COMPUTER SCINENCE AND ENGINEERING

Vision

To be excellence in education, and research in the frontier areas of computer science and engineering and produce competent, ethical and socially responsible computing graduands.

Mission

To foster computer professionals by providing innovative practices of teaching-learning process.
Facilitate the students to pursue research assisted systematic learning by dissemination of knowledge towards Computer Science and Engineering.

Programme Educational Objectives (PEO)

- PEO1 Graduates will have successful professional career by applying the knowledge of computer science and engineering.
- PEO2 Graduate will develop innovative computing skills to solve real life problems.
- PEO3 Graduate will pursue higher studies and research in the field of computer science and engineering.

Program outcomes

- **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
- **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions .

4. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
5. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

COURSE OUTCOMES:

At the end of this course, the students will be able to:

CO1-Applied control structures, arrays, and string operations to develop basic Java applications.

CO2-Applied object-oriented principles to design and implement modular Java applications using classes and constructors

CO3-Applied inheritance, polymorphism, and interfaces to develop scalable and extensible Java applications

CO4 -Applied collections, exception handling, and file I/O to develop robust and data-driven Java applications.

CO5-Applied GUI components, event handling, and synchronization techniques to create interactive and multithreaded Java applications

CONTENTS

S.No	Experiment	Page Number
1	Write Java programs using control structures (if, switch, loops)	
2	To apply arrays for structured data storage.	
3	Create a program using strings and string methods	
4	Design a class with constructors and methods (e.g., Bank Account)	
5	Implement single and multilevel inheritance (e.g., Employee Hierarchy)	
6	Use interfaces and abstract classes in a project	
7	Create a file-based student record management system	
8	Develop a program with ArrayList and exception handling.	
9	Implement multithreading using Runnable and Thread class	
10	Build a GUI login form using Swing and event handling	
11	Create a multi-threaded countdown timer with GUI	
12	12 Develop a Java application to solve the above problem using an interface.	
13	13 Develop applications to demonstrate Exception Handling and user defined exception..	
14	14 Develop a mini project for any application using Java concepts.	

15	1. Write Java programs using control structures (if, switch, loops)	
----	--	--

EXPERIMENT 1: PROGRAM TO DEFINE A STRUCTURE OF A BASIC JAVA PROGRAM.

1.A. WRITE A JAVA PROGRAM THAT DEMONSTRATES THE BASIC STRUCTURE OF A JAVA APPLICATION. YOUR PROGRAM SHOULD:

- **DEFINE A CLASS NAMED HELLOWORLD.**
- **INCLUDE THE MAIN METHOD AS THE ENTRY POINT OF THE PROGRAM.**
- **PRINT THE MESSAGE "HELLO, WORLD!" TO THE CONSOLE.**

AIM:

To write a Java program that demonstrates the basic structure of a Java application by printing "Hello, World!" to the console.

ALGORITHM:

1. Start the program.
2. Define a class named `HelloWorld`.
3. Inside the class, define the `main` method (which is the entry point of any Java application).
4. Inside the `main` method, use `System.out.println()` to display the message "Hello, World!".
5. End the program.

PROGRAM:

```
// Define the class
public class HelloWorld {

    // Main method - Entry point of the Java program
    public static void main(String[] args) {
        // Print the message to the console
        System.out.println("Hello, World!");
    }
}
```

OUTPUT

Hello, World!

1.B) WRITE A JAVA PROGRAM TO DEMONSTRATE DIFFERENT DATA TYPES AND VARIABLE DECLARATIONS. (USE INT, FLOAT, CHAR, BOOLEAN, STRING, AND DOUBLE.)

AIM:

To write a Java program that demonstrates different data types and variable declarations using `int`, `float`, `char`, `boolean`, `String`, and `double`.

ALGORITHM:

1. Start the program.
2. Declare variables of different data types:
 - `int` for integer values.
 - `float` for floating-point numbers.
 - `char` for single characters.
 - `boolean` for true/false values.
 - `String` for text data.
 - `double` for double-precision floating-point numbers.
3. Assign appropriate values to the variables.
4. Print the values of each variable using `System.out.println()`.
5. End the program.

PROGRAM:

```
public class DataTypesDemo {  
  
    public static void main(String[] args) {  
  
        // Variable declarations  
        int age = 25;  
        float temperature = 36.5f;
```

```

char grade = 'A';
boolean isJavaFun = true;
String name = "Grace";
double salary = 55000.75;

// Printing the values
System.out.println("Integer (int): " + age);
System.out.println("Float (float): " + temperature);
System.out.println("Character (char): " + grade);
System.out.println("Boolean (boolean): " + isJavaFun);
System.out.println("String (String): " + name);
System.out.println("Double (double): " + salary);
}
}

```

OUTPUT:

```

Integer (int): 25
Float (float): 36.5
Character (char): A
Boolean (boolean): true
String (String): Grace
Double (double): 55000.75

```

1.C)WRITE A PROGRAM TO PERFORM ALL ARITHMETIC OPERATIONS (ADD, SUBTRACT, MULTIPLY, DIVIDE, MODULUS) ON TWO NUMBERS.

AIM:

To write a Java program that performs all arithmetic operations—**addition**, **subtraction**, **multiplication**, **division**, and **modulus**—on two numbers.

ALGORITHM:

1. Start the program.
2. Declare two numbers (e.g., `num1` and `num2`) and assign values to them.
3. Perform the following operations:
 - Addition (+)
 - Subtraction (-)
 - Multiplication (*)
 - Division (/)
 - Modulus (%)
4. Print the results of each operation using `System.out.println()`.

5. End the program.

PROGRAM:

```
public class ArithmeticOperations {  
  
    public static void main(String[] args) {  
  
        // Declare two numbers  
        int num1 = 20;  
        int num2 = 6;  
  
        // Perform arithmetic operations  
        int sum = num1 + num2;  
        int difference = num1 - num2;  
        int product = num1 * num2;  
        int quotient = num1 / num2;  
        int remainder = num1 % num2;  
  
        // Display the results  
        System.out.println("Number 1: " + num1);  
        System.out.println("Number 2: " + num2);  
        System.out.println("Addition: " + sum);  
        System.out.println("Subtraction: " + difference);  
        System.out.println("Multiplication: " + product);  
        System.out.println("Division: " + quotient);  
        System.out.println("Modulus: " + remainder);  
    }  
}
```

OUTPUT:

Number 1: 20

Number 2: 6

Addition: 26

Subtraction: 14

Multiplication: 120

Division: 3

Modulus: 2

WRITE JAVA PROGRAMS USING CONTROL STRUCTURES (IF, SWITCH, LOOPS)

2.a)WRITE A PROGRAM TO FIND THE LARGEST OF THREE NUMBERS USING NESTED IF-ELSE.

AIM:

To write a Java program that finds the largest among three given numbers using **nested if-else** statements.

ALGORITHM:

1. Start the program.
2. Declare three variables and assign values to them.
3. Use nested if-else statements:
 - Check if the first number is greater than the second.
 - If true, compare it with the third number.
 - Otherwise, compare the second with the third.
4. Print the largest number.
5. End the program.

PROGRAM

```
public class LargestNumber {  
  
    public static void main(String[] args) {  
  
        // Declare three numbers  
        int num1 = 45;  
        int num2 = 78;  
        int num3 = 62;  
  
        // Determine the largest number using nested if-else  
        int largest;  
  
        if (num1 > num2) {  
            if (num1 > num3) {  
                largest = num1;  
            } else {  
                largest = num3;  
            }  
        } else {  
            if (num2 > num3) {  
                largest = num2;  
            }  
        }  
    }  
}
```

```

        } else {
            largest = num3;
        }
    }

    // Display the result
    System.out.println("Number 1: " + num1);
    System.out.println("Number 2: " + num2);
    System.out.println("Number 3: " + num3);
    System.out.println("The largest number is: " + largest);
}
}

```

OUTPUT

Number 1: 45

Number 2: 78

Number 3: 62

The largest number is: 78

2.b)WRITE A PROGRAM TO IMPLEMENT A SIMPLE CALCULATOR USING SWITCH-CASE.

AIM:

To write a Java program that implements a **simple calculator** using `switch-case` for performing operations like addition, subtraction, multiplication, division, and modulus based on user input.

ALGORITHM:

1. Start the program.
2. Declare two numbers and a character variable to store the operator (+, -, *, /, %).
3. Use a `switch` statement to check the operator:
 - If the operator is +, perform addition.
 - If -, perform subtraction.
 - If *, perform multiplication.
 - If /, perform division.

- If %, perform modulus.
 - For other inputs, display "Invalid operator".
4. Print the result.
 5. End the program.

PROGRAM:

```
import java.util.Scanner;

public class SimpleCalculator {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Input numbers and operator
        System.out.print("Enter first number: ");
        double num1 = sc.nextDouble();

        System.out.print("Enter second number: ");
        double num2 = sc.nextDouble();

        System.out.print("Enter an operator (+, -, *, /, %): ");
        char operator = sc.next().charAt(0);

        double result;

        // Switch-case to perform calculation
        switch (operator) {
            case '+':
                result = num1 + num2;
                System.out.println("Result: " + result);
                break;

            case '-':
                result = num1 - num2;
                System.out.println("Result: " + result);
                break;

            case '*':
                result = num1 * num2;
                System.out.println("Result: " + result);
                break;

            case '/':
```

```

        if (num2 != 0) {
            result = num1 / num2;
            System.out.println("Result: " + result);
        } else {
            System.out.println("Error: Division by zero!");
        }
        break;

    case '%':
        if (num2 != 0) {
            result = num1 % num2;
            System.out.println("Result: " + result);
        } else {
            System.out.println("Error: Division by zero!");
        }
        break;

    default:
        System.out.println("Invalid operator!");
        break;
}

sc.close();
}
}

```

OUTPUT

Enter first number: 20
 Enter second number: 4
 Enter an operator (+, -, *, /, %): *
 Result: 80.0

3.a)WRITE A JAVA PROGRAM TO PRINT THE MULTIPLICATION TABLE OF A GIVEN NUMBER USING A FOR LOOP.

AIM

To write a Java program that prints the **multiplication table** of a given number using a `for` loop.

ALGORITHM:

1. Start the program.

2. Declare a variable for the number whose multiplication table is to be printed.
3. Use a `for` loop from 1 to 10.
4. In each iteration, multiply the number by the loop variable.
5. Print the result in the format: `number x i = result`.
6. End the program.

PROGRAM:

```
import java.util.Scanner;

public class MultiplicationTable {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Get number input from the user
        System.out.print("Enter a number to print its multiplication table: ");
        int number = sc.nextInt();

        // Print multiplication table using for loop
        System.out.println("Multiplication Table of " + number + ":");
        for (int i = 1; i <= 10; i++) {
            System.out.println(number + " x " + i + " = " + (number * i));
        }

        sc.close();
    }
}
```

OUTPUT

Enter a number to print its multiplication table: 5

Multiplication Table of 5:

5 x 1 = 5

5 x 2 = 10

5 x 3 = 15

5 x 4 = 20

5 x 5 = 25

$$5 \times 6 = 30$$

$$5 \times 7 = 35$$

$$5 \times 8 = 40$$

$$5 \times 9 = 45$$

$$5 \times 10 = 50$$

TO APPLY ARRAYS FOR STRUCTURED DATA STORAGE.

4.A)STORE N NUMBERS IN AN ARRAY AND SORT IT

AIM:

To write a Java program that takes N numbers as input, stores them in an array, and prints the array in **ascending order**.

ALGORITHM:

1. Start the program.
2. Ask the user to enter the number of elements (N).
3. Create an array of size N.
4. Use a loop to read N numbers from the user into the array.
5. Use `Arrays.sort()` to sort the array.
6. Print the sorted array.
7. End the program.

PROGRAM:

```
import java.util.Scanner;
import java.util.Arrays;

public class ArraySort {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Step 1: Input number of elements
```

```

System.out.print("Enter the number of elements (N): ");
int n = sc.nextInt();

// Step 2: Declare array
int[] numbers = new int[n];

// Step 3: Read elements into array
System.out.println("Enter " + n + " numbers:");
for (int i = 0; i < n; i++) {
    numbers[i] = sc.nextInt();
}

// Step 4: Sort the array
Arrays.sort(numbers);

// Step 5: Print sorted array
System.out.println("Sorted array in ascending order:");
for (int num : numbers) {
    System.out.print(num + " ");
}

sc.close();
}
}

```

OUTPUT

```

Enter the number of elements (N): 5
Enter 5 numbers:
50
10
30
20
40
Sorted array in ascending order:
10 20 30 40 50

```

4.B) MATRIX MULTIPLICATION

AIM

To write a Java program that adds two matrices and prints the result.

ALGORITHM:

- Start the program.
- Input the number of rows and columns of the matrices.
- Declare two matrices of the same size.
- Input the elements of both matrices.
- Add corresponding elements of both matrices and store in a result matrix.
- Display the result matrix.
- End the program.

PROGRAM:

```
import java.util.Scanner;

public class MatrixAddition {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input matrix size
        System.out.print("Enter the number of rows: ");
        int rows = sc.nextInt();

        System.out.print("Enter the number of columns: ");
        int cols = sc.nextInt();

        // Declare matrices
        int[][] matrixA = new int[rows][cols];
        int[][] matrixB = new int[rows][cols];
        int[][] sum = new int[rows][cols];

        // Input Matrix A
        System.out.println("Enter elements of Matrix A:");
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                matrixA[i][j] = sc.nextInt();
            }
        }

        // Input Matrix B
        System.out.println("Enter elements of Matrix B:");
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                matrixB[i][j] = sc.nextInt();
            }
        }

        // Perform Matrix Addition
        for (int i = 0; i < rows; i++) {
```

```

        for (int j = 0; j < cols; j++) {
            sum[i][j] = matrixA[i][j] + matrixB[i][j];
        }
    }

    // Display Result
    System.out.println("Resultant Matrix after Addition:");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            System.out.print(sum[i][j] + " ");
        }
        System.out.println();
    }

    sc.close();
}
}

```

OUTPUT:

```

Enter the number of rows: 2
Enter the number of columns: 2
Enter elements of Matrix A:
1 2
3 4
Enter elements of Matrix B:
5 6
7 8
Resultant Matrix after Addition:
6    8
10   12

```

CREATE A PROGRAM USING STRINGS AND STRING METHODS

AIM

To create a Java program that takes a string as input and demonstrates common **String methods** such as:

length()
toUpperCase()
toLowerCase()
charAt()
substring()
indexOf()
replace()
equals()
trim()

ALGORITHM:



Start the program.

- Prompt the user **to enter a string and read it.**
- Display the original string.
- **Use the following string methods and display their results:**
- **length()** – to find the number of characters.
- **toUpperCase()** – to convert the string to all uppercase letters.
- **toLowerCase()** – to convert the string to all lowercase letters.
- **charAt(index)** – to display a character at a specific position (e.g., index 2).
- **substring(start, end)** – to extract a portion of the string.
- Ask the user **to enter a character and use indexOf(char)** to find its position in the string.
- Ask the user **for a word to replace and the new word, then apply replace(old, new).**
- **Use trim()** to remove leading and trailing spaces and display the trimmed string.
- Ask the user **to enter another string and use equals()** to compare the two strings.
- End the program.

PROGRAM

```
import java.util.Scanner;

public class StringMethodsDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input string
        System.out.print("Enter a string: ");
        String input = sc.nextLine();

        // Display original string
        System.out.println("\nOriginal String: " + input);
```

```

// String length
System.out.println("Length: " + input.length());

// Convert to uppercase
System.out.println("Uppercase: " + input.toUpperCase());

// Convert to lowercase
System.out.println("Lowercase: " + input.toLowerCase());

// Character at a specific index
if (input.length() >= 3) {
    System.out.println("Character at index 2: " + input.charAt(2));
}

// Substring from index 1 to 4
if (input.length() >= 5) {
    System.out.println("Substring (1 to 4): " + input.substring(1, 5));
}

// Index of a character
System.out.print("Enter a character to find its index: ");
char ch = sc.next().charAt(0);
System.out.println("Index of '" + ch + "': " + input.indexOf(ch));

// Replace a word
sc.nextLine(); // consume leftover newline
System.out.print("Enter word to replace: ");
String oldWord = sc.nextLine();
System.out.print("Enter new word: ");
String newWord = sc.nextLine();
System.out.println("String after replacement: " + input.replace(oldWord, newWord));

// Trim whitespace
System.out.println("Trimmed String: '" + input.trim() + "'");

// String comparison
System.out.print("Enter another string to compare: ");
String another = sc.nextLine();
System.out.println("Strings are equal? " + input.equals(another));

sc.close();
}
}

```

OUTPUT:

Enter a string: Hello World

Original String: Hello World

Length: 13

Uppercase: HELLO WORLD

Lowercase: hello world

Character at index 2: l

Substring (1 to 4): ello

Enter a character to find its index: o

Index of 'o': 4

Enter word to replace: World

Enter new word: Java

String after replacement: Hello Java

Trimmed String: 'Hello World'

Enter another string to compare: Hello World

Strings are equal? true

DESIGN A CLASS WITH CONSTRUCTORS AND METHODS (E.G., BANK ACCOUNT)

AIM:

To write a Java program that defines a BankAccount class with data members, constructors, and methods like deposit(), withdraw(), and displayBalance().

ALGORITHM:

1. **Start** the program.
2. **Create a class** named BankAccount with the following members:
 - Data members: name, accNo, balance
 - Constructor to initialize these values
 - Method deposit(amount) to add amount to balance
 - Method withdraw(amount) to subtract amount if sufficient balance exists
 - Method showBalance() to display account details
3. In the main method:
 - Create an object of BankAccount with name, account number, and initial balance.
 - Call showBalance() to display initial details.

- Call deposit() with a value to add to the balance.
- Call withdraw() with a value to subtract from the balance.
- Call showBalance() again to display the final balance.

4. **End** the program.

PROGRAM

```
class BankAccount {
    String name;
    int accNo;
    double balance;

    // Constructor
    BankAccount(String n, int a, double b) {
        name = n;
        accNo = a;
        balance = b;
    }

    // Deposit method
    void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: ₹" + amount);
    }

    // Withdraw method
    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: ₹" + amount);
        } else {
            System.out.println("Not enough balance!");
        }
    }

    // Show balance
    void showBalance() {
        System.out.println("Name: " + name);
        System.out.println("Account No: " + accNo);
        System.out.println("Balance: ₹" + balance);
    }
}

public class BankDemo {
    public static void main(String[] args) {
```

```

// Create account
BankAccount acc = new BankAccount("Rahul", 101, 5000);

// Show details
acc.showBalance();

// Deposit and withdraw
acc.deposit(2000);
acc.withdraw(1000);

// Final balance
acc.showBalance();
}
}

```

OUTPUT:

Name: Rahul
 Account No: 101
 Balance: ₹5000.0
 Deposited: ₹2000.0
 Withdrawn: ₹1000.0
 Name: Rahul
 Account No: 101
 Balance: ₹6000.0

IMPLEMENT SINGLE AND MULTILEVEL INHERITANCE (E.G., EMPLOYEE HIERARCHY)

AIM:

To implement single and multilevel inheritance using employee-related classes in Java.

ALGORITHM:

Start the program.

Define the base class Employee with:

- Data members: name, empId
- Method getDetails(name, empId) to accept employee details
- Method displayDetails() to display employee details

Define the derived class Manager that extends Employee (→ *Single Inheritance*):

- Data member: department

- Method setDepartment(dept) to set department name
- Method showDepartment() to display department

Define another class SeniorManager that extends Manager (→ *Multilevel Inheritance*):

- Data member: teamSize
- Method setTeamSize(size) to assign team size
- Method showTeamSize() to display the team size

In the main method:

- Create an object of SeniorManager
- Call getDetails() and displayDetails() from Employee
- Call setDepartment() and showDepartment() from Manager
- Call setTeamSize() and showTeamSize() from SeniorManager

End the program.

PROGRAM:

```
// Base class
class Employee {
    String name;
    int empId;

    void getDetails(String n, int id) {
        name = n;
        empId = id;
    }

    void displayDetails() {
        System.out.println("Employee Name: " + name);
        System.out.println("Employee ID: " + empId);
    }
}

// Derived class – Single Inheritance
class Manager extends Employee {
    String department;

    void setDepartment(String dept) {
        department = dept;
    }

    void showDepartment() {
        System.out.println("Department: " + department);
    }
}

// Further derived class – Multilevel Inheritance
class SeniorManager extends Manager {
```

```

int teamSize;

void setTeamSize(int size) {
    teamSize = size;
}

void showTeamSize() {
    System.out.println("Team Size: " + teamSize);
}
}

// Main class to test inheritance
public class EmployeeHierarchy {
    public static void main(String[] args) {
        // Multilevel inheritance in action
        SeniorManager sm = new SeniorManager();

        // From Employee class
        sm.getDetails("Anjali", 101);
        sm.displayDetails();

        // From Manager class
        sm.setDepartment("IT");
        sm.showDepartment();

        // From SeniorManager class
        sm.setTeamSize(8);
        sm.showTeamSize();
    }
}

```

OUTPUT:

Employee Name: Anjali
 Employee ID: 101
 Department: IT
 Team Size: 8

USE INTERFACES AND ABSTRACT CLASSES IN A PROJECT

AIM:

To show basic use of interface and abstract class in a simple payment example.

ALGORITHM:

Define an interface PaymentGateway with a method pay().
 Create an abstract class Payment with:

- A transaction ID
- A method to show the transaction ID
- An abstract method validate()

Create two classes:

- CreditCardPayment → uses card number
- UPIPayment → uses UPI ID
- In the main method:
- Create objects of both classes
- Call validate(), pay(), and showTransactionDetails()

End program.

PROGRAM

```
interface PaymentGateway {
    void pay(double amount);
}
```

```
abstract class Payment {
    String transactionId;

    Payment(String id) {
        transactionId = id;
    }

    void showTransactionDetails() {
        System.out.println("Transaction ID: " + transactionId);
    }

    abstract void validate();
}
```

```
class CreditCardPayment extends Payment implements PaymentGateway {
    String card;

    CreditCardPayment(String id, String card) {
        super(id);
        this.card = card;
    }

    public void pay(double amount) {
        System.out.println("Paid ₹" + amount + " using Credit Card");
    }

    void validate() {
        System.out.println("Credit Card validated: " + card);
    }
}
```

```

}

class UPIPayment extends Payment implements PaymentGateway {
    String upi;

    UPIPayment(String id, String upi) {
        super(id);
        this.upi = upi;
    }

    public void pay(double amount) {
        System.out.println("Paid ₹" + amount + " using UPI");
    }

    void validate() {
        System.out.println("UPI ID validated: " + upi);
    }
}

public class PaymentDemo {
    public static void main(String[] args) {
        CreditCardPayment cc = new CreditCardPayment("T001", "1234-5678-9012");
        UPIPayment upi = new UPIPayment("T002", "abc@upi");

        cc.showTransactionDetails();
        cc.validate();
        cc.pay(1000);

        upi.showTransactionDetails();
        upi.validate();
        upi.pay(500);
    }
}

```

OUTPUT:

Transaction ID: T001
 Credit Card validated: 1234-5678-9012
 Paid ₹1000.0 using Credit Card

Transaction ID: T002
 UPI ID validated: abc@upi
 Paid ₹500.0 using UPI

CREATE A FILE-BASED STUDENT RECORD MANAGEMENT SYSTEM

AIM:

To develop a Java program that manages student records using file handling. The system allows the user to add, view, and search student data stored in a text file, demonstrating the use of arrays, strings, loops, and file I/O operations.

ALGORITHM

1. Start the program.
2. Show menu:
 - 1. Add Student
 - 2. View Students
 - 3. Search by Roll No
 - 4. Exit
2. If user selects:
 - **Add Student** → Get details and save to file.
 - **View Students** → Read and show all records.
 - **Search** → Read file and match roll number.
3. Repeat menu until Exit is chosen.
4. End the program.

PROGRAM :

```
import java.io.*;
import java.util.*;

public class StudentManager {

    public static void main(String[] args) throws IOException {
        Scanner sc = new Scanner(System.in);
```

```

String file = "students.txt";
int choice;

do {
    System.out.println("\n1. Add 2. View 3. Search 4. Exit");
    System.out.print("Enter choice: ");
    choice = sc.nextInt();
    sc.nextLine(); // clear buffer

    if (choice == 1) {
        System.out.print("Roll No: ");
        String roll = sc.nextLine();
        System.out.print("Name: ");
        String name = sc.nextLine();
        System.out.print("Dept: ");
        String dept = sc.nextLine();

        FileWriter fw = new FileWriter(file, true);
        fw.write(roll + ", " + name + ", " + dept + "\n");
        fw.close();
        System.out.println("Student added.");
    }

    else if (choice == 2) {
        BufferedReader br = new BufferedReader(new FileReader(file));
        String line;
        System.out.println("\n-- Student List --");
        while ((line = br.readLine()) != null)
            System.out.println(line);
        br.close();
    }

    else if (choice == 3) {
        System.out.print("Enter Roll No: ");
        String search = sc.nextLine();
        BufferedReader br = new BufferedReader(new FileReader(file));
        String line;
        boolean found = false;
        while ((line = br.readLine()) != null) {
            if (line.startsWith(search + ",")) {
                System.out.println("Found: " + line);
                found = true;
                break;
            }
        }
    }
} while (choice != 4);

```

```

        System.out.println("Not found.");
        br.close();
    }

    } while (choice != 4);

    sc.close();
}
}

```

OUTPUT

1. Add 2. View 3. Search 4. Exit

Enter choice: 1

Roll No: 101

Name: Arjun

Dept: CSE

Student added.

Enter choice: 2

-- Student List --

101,Arjun,CSE

Enter choice: 3

Enter Roll No: 101

Found: 101,Arjun,CSE

DEVELOP A PROGRAM WITH ARRAYLIST AND EXCEPTION HANDLING.

AIM

To develop a Java program using ArrayList and basic exception handling to store and display student names.

ALGORITHM

- Start the program.
- Create an ArrayList to store names.
- Use a loop with a menu:
 - Add a name
 - View all names
 - Exit
- Use try-catch to handle input errors.
- End the program.

PROGRAM

```
import java.util.*;

public class StudentList {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<String> names = new ArrayList<>();
        int choice;

        do {
            System.out.println("\n1. Add Name");
            System.out.println("2. View Names");
            System.out.println("3. Exit");
            System.out.print("Enter choice: ");

            try {
                choice = sc.nextInt();
                sc.nextLine(); // clear buffer

                if (choice == 1) {
                    System.out.print("Enter name: ");
                    String name = sc.nextLine();
                    names.add(name);
                    System.out.println("Name added.");
                } else if (choice == 2) {
                    System.out.println("\nStudent Names:");
                    for (String n : names)
                        System.out.println(n);
                } else if (choice == 3) {
                    System.out.println("Exiting...");
                } else {
                    System.out.println("Invalid choice.");
                }

            } catch (InputMismatchException e) {
                System.out.println("Please enter a valid number!");
                sc.nextLine(); // clear invalid input
                choice = 0; // reset choice to continue loop
            }

        } while (choice != 3);

        sc.close();
    }
}
```

```
}
```

OUTPUT

```
1. Add Name
2. View Names
3. Exit
Enter choice: 1
Enter name: Ravi
Name added.
```

```
Enter choice: 2
Student Names:
Ravi
```

```
Enter choice: x
Please enter a valid number!
```

IMPLEMENT MULTITHREADING USING RUNNABLE AND THREAD CLASS

AIM:

To implement multithreading in Java using both the Thread class and the Runnable interface.

ALGORITHM:

- Create a class that extends Thread and overrides run().
- Create another class that implements Runnable and overrides run().
- Create objects of both classes.
- Start both threads using start() method.
- The run() methods will execute in parallel.

PROGRAM

```
// Thread class method
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread (Thread class): " + i);
        }
    }
}
```

```
// Runnable interface method
class MyRunnable implements Runnable {
    public void run() {
```

```

        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread (Runnable): " + i);
        }
    }
}

public class MultithreadDemo {
    public static void main(String[] args) {
        // Using Thread class
        MyThread t1 = new MyThread();

        // Using Runnable interface
        MyRunnable r1 = new MyRunnable();
        Thread t2 = new Thread(r1); // wrap in Thread object

        // Start both threads
        t1.start();
        t2.start();
    }
}

```

OUTPUT

```

Thread (Thread class): 1
Thread (Runnable): 1
Thread (Thread class): 2
Thread (Runnable): 2
...

```

BUILD A GUI LOGIN FORM USING SWING AND EVENT HANDLING

AIM:

To create a basic GUI-based login form using Java Swing and handle button click events.

ALGORITHM :

- Create a class that extends JFrame and implements ActionListener.
- Add Swing components: JLabel, JTextField, JPasswordField, and JButton.
- Set layout and position for components.
- Add an ActionListener to the Login button.
- In actionPerformed(), check if the username and password match predefined values.
- Show a message dialog for success or failure.

PROGRAM

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

```

```

public class LoginForm extends JFrame implements ActionListener {
    // Components
    JLabel labelUser, labelPass;
    JTextField textUser;
    JPasswordField textPass;
    JButton btnLogin;

    // Constructor
    LoginForm() {
        // Frame setup
        setTitle("Login Form");
        setSize(300, 200);
        setLayout(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Username label & field
        labelUser = new JLabel("Username:");
        labelUser.setBounds(30, 30, 80, 25);
        add(labelUser);

        textUser = new JTextField();
        textUser.setBounds(120, 30, 130, 25);
        add(textUser);

        // Password label & field
        labelPass = new JLabel("Password:");
        labelPass.setBounds(30, 70, 80, 25);
        add(labelPass);

        textPass = new JPasswordField();
        textPass.setBounds(120, 70, 130, 25);
        add(textPass);

        // Login button
        btnLogin = new JButton("Login");
        btnLogin.setBounds(100, 110, 80, 30);
        btnLogin.addActionListener(this);
        add(btnLogin);

        setVisible(true);
    }

    // Event Handling
    public void actionPerformed(ActionEvent e) {
        String user = textUser.getText();
    }
}

```

```

String pass = new String(textPass.getPassword());

if (user.equals("admin") && pass.equals("1234")) {
    JOptionPane.showMessageDialog(this, "Login Successful!");
} else {
    JOptionPane.showMessageDialog(this, "Invalid Credentials!");
}
}

// Main method
public static void main(String[] args) {
    new LoginForm();
}
}

```

OUTPUT :

A small window will pop up with:

Username & password input fields

Login button

A message dialog on clicking the button (based on input)

CREATE A MULTI-THREADED COUNTDOWN TIMER WITH GUI

AIM:

To develop a GUI-based countdown timer in Java using Swing and multithreading.

ALGORITHM:

- Create a class that extends JFrame.
- Add components: JLabel, JTextField, and JButton.
- On button click, create and start a new thread.
- Inside the thread, decrease the number every second.
- Update the label after each second using SwingUtilities.invokeLater() or Swing Timer.

PROGRAM

```
import javax.swing.*;
```

```

import java.awt.event.*;

public class CountdownTimer extends JFrame implements ActionListener {
    JTextField inputField;
    JLabel timerLabel;
    JButton startButton;

    CountdownTimer() {
        setTitle("Countdown Timer");
        setSize(300, 150);
        setLayout(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JLabel inputLabel = new JLabel("Enter seconds:");
        inputLabel.setBounds(20, 20, 100, 25);
        add(inputLabel);

        inputField = new JTextField();
        inputField.setBounds(130, 20, 120, 25);
        add(inputField);

        timerLabel = new JLabel("Time left: ");
        timerLabel.setBounds(20, 60, 200, 25);
        add(timerLabel);

        startButton = new JButton("Start");
        startButton.setBounds(100, 90, 80, 30);
        startButton.addActionListener(this);
        add(startButton);

        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        int seconds;
        try {
            seconds = Integer.parseInt(inputField.getText());
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Please enter a valid number.");
            return;
        }

        // Start countdown in a new thread
        Thread t = new Thread() -> {
            for (int i = seconds; i >= 0; i--) {
                int timeLeft = i;
            }
        };
    }
}

```

```

        SwingUtilities.invokeLater(() -> timerLabel.setText("Time left: " + timeLeft + " sec"));
    try {
        Thread.sleep(1000);
    } catch (InterruptedException ex) {
        ex.printStackTrace();
    }
}
SwingUtilities.invokeLater(() -> JOptionPane.showMessageDialog(this, "Time's up!"));
});
t.start();
}

public static void main(String[] args) {
    new CountdownTimer();
}
}

```

DEVELOP A JAVA APPLICATION TO SOLVE THE ABOVE PROBLEM USING AN INTERFACE.

AIM:

To build a basic countdown timer using Java Swing and implement logic through an interface.

ALGORITHM

- Create an interface Countdown with a method startCountdown().
- Design a GUI with a text field, label, and button.
- When the user clicks the button, start the countdown in a thread.

PROGRAM

```

import javax.swing.*;
import java.awt.event.*;

// Step 1: Create an interface
interface Countdown {
    void startCountdown(int time);
}

// Step 2: Main class
public class SimpleCountdown extends JFrame implements Countdown, ActionListener {
    JTextField timeInput;
    JLabel display;
    JButton startBtn;
}

```

```

SimpleCountdown() {
    setTitle("Timer");
    setSize(250, 150);
    setLayout(null);
    setDefaultCloseOperation(EXIT_ON_CLOSE);

    timeInput = new JTextField();
    timeInput.setBounds(70, 20, 100, 25);
    add(timeInput);

    display = new JLabel("Time left:");
    display.setBounds(70, 50, 150, 25);
    add(display);

    startBtn = new JButton("Start");
    startBtn.setBounds(80, 80, 80, 25);
    startBtn.addActionListener(this);
    add(startBtn);

    setVisible(true);
}

// Step 3: Implement the interface method
public void startCountdown(int time) {
    new Thread(() -> {
        for (int i = time; i >= 0; i--) {
            int current = i;
            SwingUtilities.invokeLater(() -> display.setText("Time left: " + current));
            try {
                Thread.sleep(1000);
            } catch (Exception e) {}
        }
        SwingUtilities.invokeLater(() -> JOptionPane.showMessageDialog(this, "Done!"));
    }).start();
}

// Step 4: Handle button click
public void actionPerformed(ActionEvent e) {
    try {
        int time = Integer.parseInt(timeInput.getText());
        startCountdown(time);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Enter number only");
    }
}

```

```

    public static void main(String[] args) {
        new SimpleCountdown();
    }
}

```

DEVELOP APPLICATIONS TO DEMONSTRATE EXCEPTION HANDLING AND USER DEFINED EXCEPTION.

AIM:

To develop a Java program that demonstrates both built-in exception handling and a user-defined exception.

ALGORITHM

- Create a user-defined exception class InvalidAgeException.
- In the main program, ask the user to enter their age.
- If the age is less than 18, throw InvalidAgeException.
- Handle the exception using a try-catch block.
- Also demonstrate handling of built-in exceptions like InputMismatchException.

PROGRAM

```

import java.util.*;

// Step 1: Define user-defined exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class ExceptionDemo {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            // Step 2: Get age input
            System.out.print("Enter your age: ");
            int age = sc.nextInt();

            // Step 3: Check condition
            if (age < 18) {
                throw new InvalidAgeException("Age must be 18 or above.");
            }
        }
    }
}

```

```
        System.out.println("You are eligible to vote!");
    } catch (InvalidAgeException e) {
        System.out.println("Custom Exception Caught: " + e.getMessage());
    } catch (InputMismatchException e) {
        System.out.println("Built-in Exception: Please enter a valid number.");
    } finally {
        System.out.println("Program ended.");
        sc.close();
    }
}
}
```

OUTPUT

Enter your age: 20
You are eligible to vote!
Program ended.